

45-Day QA Upskilling Calendar

2 hours a day - Manual QA to Automation Engineer

Tool-agnostic structure: works for Selenium, Playwright, Cypress, WebdriverIO, etc.

By Sreenidhi Rajakrishnan

Author and curator of this 45-day program

How to use this calendar

Block 2 hours every day on your calendar. Hour 1 is theory and reading. Hour 2 is hands-on. Push your work to a single GitHub repository called **qa-upskilling-45** so by Day 45 you have a public portfolio. If a day feels heavy, do not skip it -- split it across two evenings. Consistency beats intensity.

1	1 - 10	Programming Foundations (OOP, strings, collections, design patterns, SOLID)	20
2	11 - 20	Web Tech & Automation Tool (locators, waits, alerts, frames, windows, shadow DOM)	20
3	21 - 28	Test Framework (POM, folder structure, data-driven, reporting, parallel)	16
4	29 - 33	API Testing (REST, Postman, RestAssured/Requests, schema, mocking)	10
5	34 - 38	CI/CD & DevOps (Git, Docker, GitHub Actions/Jenkins, notifications)	10
6	39 - 42	GenAI for QA (prompting, Copilot/Cursor, AI test data, AI-native tools)	8
7	43 - 45	Specializations + Capstone (perf, security, a11y, portfolio, interview)	6

Daily structure

Hour 1 (Learn) -- read docs, watch one focused tutorial, take short notes. **Hour 2 (Practice)** -- code in your repo, no copy-paste. **Deliverable** -- the artifact you push to GitHub at end of day. **Pro tip** -- a small habit or shortcut to make the topic stick.

Pick **one** language (Java / Python / JS-TS / C#) and **one** automation tool on Day 1 and stay with them through Day 45. Switching mid-plan resets your muscle memory.

PHASE 1 | Days 1 - 10 | Programming Foundations

Goal: become comfortable enough with one language that you can write, read, and debug ~50 lines without help. Cover OOP, strings, collections, exceptions, and the design patterns/SOLID principles used in test frameworks.

Day 1 · Dev Environment & Version Control

Hour 1 (Learn)	Install your IDE (VS Code or IntelliJ/PyCharm), language SDK, Git. Create a GitHub account. Learn clone, add, commit, push, pull, status, log.
Hour 2 (Practice)	Create the qa-upskilling-45 repo. Push a Hello World file. Add a README that lists your goals.
Deliverable	Public GitHub repo with first commit and README.
Pro tip	Use a .gitignore from gitignore.io for your language so you do not commit IDE junk.

Day 2 · Variables, Data Types & Operators

Hour 1 (Learn)	Primitives vs. reference types, casting/auto-boxing, arithmetic / logical / relational operators, null vs. None vs. undefined.
Hour 2 (Practice)	Write 5 mini scripts: BMI calculator, temperature converter, simple interest, age in days, tip splitter.
Deliverable	5 small scripts in <i>/day02</i> folder.
Pro tip	Print intermediate values while you learn -- it is the cheapest debugger.

Day 3 · Conditionals & Loops

Hour 1 (Learn)	if / else if / else, switch / match, for, while, do-while, break, continue, nested loops.
Hour 2 (Practice)	Solve FizzBuzz, prime checker, factorial, palindrome, multiplication table.
Deliverable	5 solutions committed in <i>/day03</i> .
Pro tip	Re-solve each problem two different ways (e.g., for-loop vs. while-loop) to build flexibility.

Day 4 · Functions / Methods

Hour 1 (Learn)	Definition, parameters, return values, default args, scope, pass-by-value vs. reference, recursion basics.
Hour 2 (Practice)	Refactor Day 2 and Day 3 code into reusable functions. Write 3 utility functions: isEven, max-of-three, reverseString.
Deliverable	<i>/utils</i> file with reusable functions, called from old scripts.
Pro tip	If a function is more than 20 lines, split it. Test frameworks reward small functions.

Day 5 · Strings & String Operations

Hour 1 (Learn)	String immutability, length, indexOf, substring, split, join, replace, trim, format / f-strings / template literals, basic regex (\d, \w, +, *, ^, \$).
Hour 2 (Practice)	Solve: reverse a string, count vowels, check anagram, capitalize words, character frequency map.
Deliverable	<i>StringUtils</i> module with 5 methods + tests in <i>/day05</i> .
Pro tip	Strings are 60% of QA scripting -- learn one regex flavor (PCRE) really well.

Day 6 · Collections / Data Structures

Hour 1 (Learn)	Array vs. List/ArrayList, Set, Map / Dictionary / HashMap, iteration patterns, common methods (add, contains, remove, sort).
Hour 2 (Practice)	Solve: sum array, find duplicates, group words by length, sort by value, two-sum.
Deliverable	Collection-based solutions in <i>/day06</i> .
Pro tip	Pick the right collection: List for order, Set for uniqueness, Map for lookups.

Day 7 · OOP Part 1 -- Classes & Encapsulation

Hour 1 (Learn)	Classes, objects, constructors, fields, methods, public/private/protected, getters/setters, encapsulation.
Hour 2 (Practice)	Build a <i>BankAccount</i> class (deposit, withdraw, balance) and a <i>Person</i> class. Write a small driver to use them.
Deliverable	2 OOP classes + driver in <i>/day07</i> .
Pro tip	Make fields private by default. Expose only what callers need.

Day 8 · OOP Part 2 -- Inheritance, Polymorphism, Abstraction, Interfaces

Hour 1 (Learn)	extends / inherits, method override, super, abstract classes vs. interfaces, polymorphism, composition over inheritance.
Hour 2 (Practice)	Build <i>Vehicle</i> -> <i>Car</i> / <i>Bike</i> / <i>Truck</i> hierarchy. Define a <i>Drivable</i> interface and have classes implement it.
Deliverable	Mini OOP project in <i>/day08</i> .
Pro tip	Page Object Model in test frameworks IS OOP -- nail this day.

Day 9 · Exceptions, File I/O & Logging

Hour 1 (Learn)	try / catch / finally, throw / raise, custom exception classes, read and write text/CSV files, basic logging library setup.
Hour 2 (Practice)	Build a CSV reader that returns a list of rows; log errors to a file when the CSV is malformed.
Deliverable	CSV reader utility + log file in <i>/day09</i> .
Pro tip	Never swallow exceptions silently -- log them with context, then rethrow if you cannot handle.

Day 10 · Design Patterns + SOLID for Testers

Hour 1 (Learn)	Singleton (config / driver), Factory (driver factory), Builder (test data), Page Object (preview). SOLID principles with QA examples.
Hour 2 (Practice)	Implement Singleton config-reader and a Factory that returns a driver based on browser name (pseudocode is fine).
Deliverable	Pattern examples + 1-page SOLID notes in <i>/day10</i> .
Pro tip	You will use Singleton, Factory, and Page Object every single day in real frameworks.

PHASE 2 | Days 11 - 20 | Web Tech & Automation Tool Fundamentals

Goal: install your chosen automation tool and master locators, synchronization, and every tricky element type (alerts, frames, windows, shadow DOM, file uploads). Practice on demo sites: the-internet.herokuapp.com, saucedemo.com, demoqa.com, automationexercise.com.

Day 11 · HTML, CSS & DOM Refresher

Hour 1 (Learn)	HTML structure & semantic tags, attributes, CSS selectors (descendant, child, attribute), the DOM tree, DevTools.
Hour 2 (Practice)	Open 3 real sites in DevTools. For each, write 10 selectors that uniquely target a button, input, or link.
Deliverable	Selectors notebook (markdown) committed to repo.
Pro tip	Master DevTools \$ (querySelector) and \$\$ (querySelectorAll) in the console.

Day 12 · Automation Tool Setup & First Script

Hour 1 (Learn)	Install Selenium / Playwright / Cypress (pick ONE). Browser drivers / installers. Project skeleton. Run a sample test.
Hour 2 (Practice)	Write 'open google.com -> type query -> assert title contains query'. Run it locally.
Deliverable	Hello-world automation script in /day12, runs green.
Pro tip	Pin your tool version. Random version drift is the #1 cause of 'works on my machine' bugs.

Day 13 · Locators -- Basics

Hour 1 (Learn)	ID, name, class, tag, link text, partial link text. When each is appropriate. Stability hierarchy.
Hour 2 (Practice)	Automate login on saucedemo.com using ID and name locators. Verify successful login with an assertion.
Deliverable	Login test in /day13.
Pro tip	Prefer test-id attributes (data-testid) when devs cooperate -- they survive UI rewrites.

Day 14 · Advanced Locators -- CSS & XPath

Hour 1 (Learn)	CSS combinators (>, +, ~), pseudo-classes (:nth-child, :not). XPath axes (parent, following-sibling), functions (contains, starts-with, text, normalize-space).
Hour 2 (Practice)	Locate 15 hard elements (table cell by row+column, button after a label, dynamic IDs) on demoqa.com.
Deliverable	Locator cheatsheet markdown in /day14.
Pro tip	If your XPath has more than 3 indexes, it is fragile. Refactor.

Day 15 · Synchronization -- Implicit, Explicit, Fluent Waits

Hour 1 (Learn)	Why Thread.sleep is evil. Implicit waits, explicit waits, fluent waits, expected conditions, polling intervals.
Hour 2 (Practice)	Take a flaky test and remove every sleep. Replace with explicit waits keyed to element state.
Deliverable	Refactored stable test in <i>/day15</i> .
Pro tip	Never mix implicit and explicit waits in the same suite -- timings multiply.

Day 16 · Web Element Interactions

Hour 1 (Learn)	click, type/sendKeys, clear, getText, getAttribute, dropdowns (Select / native), checkboxes, radio buttons, isDisplayed/isEnabled/isSelected.
Hour 2 (Practice)	Automate a multi-field form end-to-end (text, dropdown, radio, checkbox, submit). Validate confirmation page.
Deliverable	Form automation test in <i>/day16</i> .
Pro tip	After every action, assert the resulting state -- silent failures are the worst kind.

Day 17 · Alerts, Frames & iFrames

Hour 1 (Learn)	JS alerts (accept, dismiss, getText, sendKeys). switchTo.frame by index/name/element. Nested frames. switchTo.defaultContent.
Hour 2 (Practice)	On the-internet.herokuapp.com solve all three alert challenges and the nested frames page.
Deliverable	Alerts and iframe tests in <i>/day17</i> .
Pro tip	Always switch back to defaultContent before moving on, or your next locator will mysteriously fail.

Day 18 · Window & Tab Handles

Hour 1 (Learn)	getWindowHandle (current), getWindowHandles (all), switching context, opening a new tab/window, closing children, returning to parent.
Hour 2 (Practice)	Automate: click a link that opens a new tab, assert content there, close it, assert you are back on parent.
Deliverable	Multi-window test in <i>/day18</i> .
Pro tip	Store the parent handle in a variable BEFORE opening children -- order is not guaranteed.

Day 19 · Advanced Actions -- Mouse, Keyboard, Drag-Drop, JS Executor

Hour 1 (Learn)	Actions class: hover, right-click, double-click, drag-and-drop, keyDown/keyUp. JavaScriptExecutor for scrolling, hidden elements, attribute reads.
Hour 2 (Practice)	Automate a hover menu (e.g. amazon.in categories) and a drag-drop demo (jqueryui).
Deliverable	Actions tests in <i>/day19</i> .
Pro tip	JS executor is your escape hatch. Use it last, never first.

Day 20 · Shadow DOM, File Upload, Screenshots, Cookies

Hour 1 (Learn)	Pierce shadow roots (shadowRoot in JS / built-in helpers in Playwright/Cypress). Upload files via sendKeys to input[type=file]. Capture screenshots. Read/set cookies.
Hour 2 (Practice)	Automate a shadow DOM demo + file upload + take screenshots after each step. Save them under <i>/screenshots</i> .
Deliverable	Advanced interactions test in <i>/day20</i> .
Pro tip	Shadow DOM is everywhere in modern UI kits (Salesforce, Stencil, Lit). Learn it once, save weeks later.

PHASE 3 | Days 21 - 28 | Test Framework Building

Goal: graduate from scripts to a real framework. Test runner, Page Object Model, clean folder structure, data-driven, reporting, parallel execution. End the phase with a working mini-framework you can demo in interviews.

Day 21 · Test Runner & Assertions

Hour 1 (Learn)	TestNG / JUnit / PyTest / Mocha basics: @Test, @BeforeMethod / @BeforeEach, @AfterMethod, fixtures, soft vs. hard asserts, groups/tags.
Hour 2 (Practice)	Convert your Day 13, 16, and 18 scripts into proper test cases with assertions. Run them all from the runner.
Deliverable	10 organized tests in <i>/tests</i> .
Pro tip	One assertion per test is too few; ten is too many. Aim for 1-3.

Day 22 · Page Object Model (POM)

Hour 1 (Learn)	POM concepts: page = class, elements = locator fields, actions = methods returning either void or the next page object (fluent).
Hour 2 (Practice)	Refactor login + form tests into <i>LoginPage</i> , <i>HomePage</i> , <i>FormPage</i> classes.
Deliverable	POM-based tests in <i>/pages</i> and <i>/tests</i> .
Pro tip	If the same locator appears in 2+ tests, it belongs in a page object.

Day 23 · Folder Structure & Config Management

Hour 1 (Learn)	Standard layout: <i>src/main</i> for pages/utils, <i>src/test</i> for tests, <i>resources/</i> for config, <i>reports/</i> , <i>screenshots/</i> . Externalize URL, browser, credentials to config files / env vars.
Hour 2 (Practice)	Create the project skeleton + ConfigReader + BaseTest class with setUp/tearDown.
Deliverable	Reusable project skeleton in <i>/framework</i> .
Pro tip	Never hardcode environment URLs. Future-you will thank present-you.

Day 24 · Data-Driven Testing

Hour 1 (Learn)	DataProvider (TestNG) / @ParameterizedTest (JUnit) / @pytest.mark.parametrize / data() in Mocha. Read CSV, JSON, Excel. Faker for synthetic data.
Hour 2 (Practice)	Run login test with 5 datasets from CSV (valid, invalid pwd, locked user, empty fields, SQLi).
Deliverable	Data-driven test suite in <i>/data</i> + tests.
Pro tip	Negative cases beat positive ones in interviews -- design for them deliberately.

Day 25 · Reporting -- Allure / Extent / built-in HTML

Hour 1 (Learn)	Install reporter, attach screenshots and steps, severity, environment metadata. Browse the generated report.
Hour 2 (Practice)	Generate the report on your full suite. Mark steps with @Step / annotate. Open and analyze a failing test.
Deliverable	Allure or Extent report committed under <i>/reports</i> .
Pro tip	Tag tests by feature in the report -- recruiters love seeing organized output.

Day 26 · Logging & Failure Diagnostics

Hour 1 (Learn)	Log4j2 / SLF4J / python logging / debug. Log levels (TRACE, DEBUG, INFO, WARN, ERROR). Listener / Hook to capture screenshot + DOM on failure.
Hour 2 (Practice)	Add a listener that, on failure, saves screenshot + page source + browser console log.
Deliverable	Failure-handling utility committed to <i>/utils</i> .
Pro tip	INFO for what happened, DEBUG for why, ERROR only for actual failures.

Day 27 · Cross-Browser & Parallel Execution

Hour 1 (Learn)	Browser matrix (Chrome, Firefox, Edge, WebKit). Selenium Grid / Playwright projects / Cypress parallel via cloud. Thread-safe driver pattern.
Hour 2 (Practice)	Run your suite in parallel across 2-3 browsers. Capture timing improvement.
Deliverable	Cross-browser run report + before/after timings.
Pro tip	Driver instance must be ThreadLocal in Java. Race conditions ruin parallel runs.

Day 28 · Mini Capstone -- End-to-End Framework

Hour 1 (Learn)	Plan: 5 tests across 2 pages, POM, data-driven, reporting, config, listeners, parallel.
Hour 2 (Practice)	Build it end-to-end, run green, push to GitHub with a clear README.
Deliverable	Mini framework v1.0 in <i>/framework</i> with README + screenshots of report.
Pro tip	Treat the README as a sales pitch -- it is what hiring managers actually read first.

PHASE 4 | Days 29 - 33 | API Testing

Goal: become equally comfortable testing without a UI. HTTP, REST, JSON, Postman, then automated API tests in code (RestAssured / Requests / Supertest / Playwright APIRequest), schema validation, and mocking.

Day 29 · HTTP, REST & JSON Fundamentals

Hour 1 (Learn)	HTTP methods (GET/POST/PUT/PATCH/DELETE), status codes (2xx/3xx/4xx/5xx), headers, query vs. path vs. body params, REST principles, JSON structure.
Hour 2 (Practice)	Hit reqres.in and jsonplaceholder.typicode.com from browser + curl. Document each response.
Deliverable	<i>api-notes.md</i> with annotated requests/responses.
Pro tip	Status codes are language. 200 vs. 201 vs. 204 each say something different -- learn the dialect.

Day 30 · Postman Mastery

Hour 1 (Learn)	Collections, environments, variables, pre-request scripts, test scripts, chained requests, Newman (CLI runner).
Hour 2 (Practice)	Build a 6-request collection: signup -> login -> create resource -> read -> update -> delete. Use environment variables for token chaining.
Deliverable	Exported Postman collection (.json) in <i>/api/postman</i> .
Pro tip	<i>pm.test()</i> assertions inside Postman = real automated tests, not just exploration.

Day 31 · API Automation in Code

Hour 1 (Learn)	RestAssured (Java) / requests + pytest (Python) / Supertest (JS) / Playwright APIRequest (any). Send requests, parse responses, assert status/body.
Hour 2 (Practice)	Convert your Postman collection to automated tests. Run them via the test runner.
Deliverable	API test suite in <i>/api/tests</i> .
Pro tip	Keep API tests in the SAME repo as UI tests -- shared utils, shared CI.

Day 32 · Auth, Schema Validation & Negative Cases

Hour 1 (Learn)	Basic / Bearer / OAuth2 / API key. JSON schema validation (json-schema-validator / jsonschema / ajv). Response time assertions. Negative tests (bad payload, missing token, expired token).
Hour 2 (Practice)	Add JSON schema validation to 3 endpoints. Add 5 negative tests.
Deliverable	Hardened API suite + <i>/schemas</i> folder.
Pro tip	401 vs. 403 confusion is a classic interview gotcha. Know the difference.

Day 33 · Mocking & Contract Testing Intro

Hour 1 (Learn)	WireMock / Mockoon / msw basics. Why mock? Contract testing (Pact) at a conceptual level: consumer-driven contracts.
Hour 2 (Practice)	Stand up a Mockoon mock for a downstream service and point one of your tests at it.
Deliverable	Mock config + mocked test scenario in <i>/api/mocks</i> .
Pro tip	Mocking is a shift-left superpower -- your team will love you for unblocking them.

PHASE 5 | Days 34 - 38 | CI/CD & DevOps for QA

Goal: tests must run on every push, on a schedule, in containers, and report results back to the team. Cover Git in depth, Docker, GitHub Actions or Jenkins, and notifications.

Day 34 · Git Advanced

Hour 1 (Learn)	Branches, merge vs. rebase, fast-forward, conflict resolution, pull request flow, code review etiquette, .gitignore, signed commits.
Hour 2 (Practice)	Open a feature branch, push it, raise a PR against main, self-review and merge. Tag a release v0.1.
Deliverable	PR history visible in repo + tag.
Pro tip	<i>git rebase -i</i> to clean up commits before opening a PR -- shows craftsmanship.

Day 35 · Docker for Testing

Hour 1 (Learn)	Image vs. container, Dockerfile basics, docker-compose, official Selenium / Playwright images, headless run.
Hour 2 (Practice)	Containerize one of your tests. Build the image, run inside Docker, view logs.
Deliverable	Dockerfile + commands documented in README.
Pro tip	Docker eliminates 'works on my machine'. CI environments are containers anyway.

Day 36 · GitHub Actions / Jenkins -- Setup

Hour 1 (Learn)	Workflow YAML / Jenkinsfile syntax. Triggers (push, PR, schedule). Runner / agent. Steps. Caching.
Hour 2 (Practice)	Create a workflow that runs your test suite on every push to main. Watch it go green.
Deliverable	<i>.github/workflows/ci.yml</i> (or Jenkinsfile) committed.
Pro tip	Start with the simplest possible workflow. Add complexity only when something hurts.

Day 37 · Pipeline as Code & Scheduled Runs

Hour 1 (Learn)	Stages, matrix builds (Chrome + Firefox + Edge), secrets, artifact upload, cron schedules, manual triggers.
Hour 2 (Practice)	Add a nightly run (2 AM) that uploads the Allure / HTML report as an artifact.
Deliverable	Updated workflow with schedule + artifact upload.
Pro tip	Use repository secrets for credentials. NEVER hardcode tokens in YAML.

Day 38 · Test Reporting in CI + Notifications

Hour 1 (Learn)	Publish reports as GitHub Pages or Jenkins HTML Publisher. Slack / Teams / email notifications on failure.
Hour 2 (Practice)	Wire a Slack incoming webhook. Make the pipeline post a message on failure with the report link.
Deliverable	Notification working in CI + screenshot in README.
Pro tip	Loud failures, silent successes. Notification fatigue kills CI culture.

PHASE 6 | Days 39 - 42 | GenAI for QA

Goal: 10x your output with AI without losing your engineering judgment. Prompt engineering, Copilot/Cursor, AI for test data and triage, AI-native testing tools.

Day 39 · GenAI Fundamentals & Prompt Engineering for QA

Hour 1 (Learn)	How LLMs work at a high level. Prompt patterns: role + context + constraints + examples + format. Hallucinations, grounding, temperature.
Hour 2 (Practice)	Use ChatGPT / Claude to generate 20 test cases (positive + negative + edge) from a 1-page requirements doc.
Deliverable	<code>/ai/test-cases.md</code> + the prompts you used.
Pro tip	Save your prompts in version control. Prompts are code.

Day 40 · GitHub Copilot & Cursor for Test Code

Hour 1 (Learn)	Install Copilot or Cursor. Inline suggestions, chat, edit-in-place. Comment-driven code generation patterns. When to accept and when to reject.
Hour 2 (Practice)	Generate page objects + tests for a new page using AI assist. Refactor and review every line.
Deliverable	AI-assisted test code in <code>/day40</code> , with a CHANGELOG of what you accepted vs. fixed.
Pro tip	Always read what the AI wrote. Confidently wrong code is worse than missing code.

Day 41 · AI for Test Data, Bug Triage & Log Analysis

Hour 1 (Learn)	Prompts that produce realistic test data (PII-safe). Summarize failing test reports. Triage logs to suggest probable cause.
Hour 2 (Practice)	Use AI to: (1) generate 50 realistic users with edge cases, (2) summarize a failing test run into a 1-paragraph triage note.
Deliverable	Generated dataset + triage note in <code>/ai/data</code> and <code>/ai/triage</code> .
Pro tip	Mask real data before pasting into any model. Treat prompts like git history -- public, forever.

Day 42 · AI-Native Testing Tools

Hour 1 (Learn)	Tour: Testim, Mabl, KaneAI (LambdaTest), BrowserStack AI, Functionize. Self-healing locators. Auto test generation from user flows.
Hour 2 (Practice)	Sign up for one free tier. Record a flow, run it, intentionally break a locator and watch self-heal.
Deliverable	Comparison notes + screenshots in <code>/ai/tools</code> .
Pro tip	AI tools complement code-based frameworks; they rarely replace them in serious shops.

PHASE 7 | Days 43 - 45 | Specializations + Capstone

Goal: brush against the adjacent disciplines (performance, security, accessibility, mobile) and ship a portfolio + interview-ready demo.

Day 43 · Performance Testing Intro

Hour 1 (Learn)	JMeter, k6, Gatling at a conceptual level. Load vs. stress vs. soak. Key metrics: throughput, p95/p99 latency, error rate, ramp-up.
Hour 2 (Practice)	Run a 50-virtual-user load test against a public API (e.g., reqres.in) using k6 or JMeter. Capture the report.
Deliverable	Performance test script + report in <i>/perf</i> .
Pro tip	Always test the system you can break -- never run load tests against production without permission.

Day 44 · Security, Accessibility & Mobile Primer

Hour 1 (Learn)	OWASP Top 10 (high-level). OWASP ZAP baseline scan. axe-core for a11y. Appium and Espresso/XCUITest for mobile (concept only).
Hour 2 (Practice)	Run an axe-core scan and a ZAP baseline scan against a demo site. Document the top 5 findings each.
Deliverable	Findings summary in <i>/sec-a11y</i> .
Pro tip	Accessibility tests catch real bugs and real lawsuits. Wedge axe-core into every UI suite.

Day 45 · Capstone, Portfolio & Interview Prep

Hour 1 (Learn)	Polish the framework README, record a 3-minute demo video, write a LinkedIn post about your 45-day journey, prepare a 2-minute project pitch.
Hour 2 (Practice)	Practice 10 common SDET interview questions out loud. Apply to 3 jobs.
Deliverable	Final portfolio: repo + README + demo video + LinkedIn post + 3 applications submitted.
Pro tip	Your repo + a clear README + a 3-min demo beats most resumes. Put the link at the top of your CV.

Habits that decide whether this works

- **Same time, every day.** Block the 2 hours on your calendar in advance. Treat it like a stand-up.
- **One repo, every commit.** qa-upskilling-45 is your story. Recruiters will scroll the commit graph.
- **Type the code, do not paste.** Muscle memory is what survives interview pressure.
- **Teach to learn.** Every Sunday, write a 200-word LinkedIn post on what you learned that week.
- **Ask, do not stall.** Stuck more than 30 minutes? Ask in a community (Test Automation University Slack, r/QualityAssurance, Ministry of Testing).
- **Refactor weekly.** On Day 7/14/21/28/35/42, spend Hour 2 cleaning up earlier code. Real engineers refactor.
- **One language, one tool, one CI.** Do not switch stacks until Day 45.
- **Ship, do not perfect.** Day 45 framework should run, even if it is small. Done beats perfect.

By Day 45 you will have

- A public GitHub repo with 45+ commits and a real test framework.
- A working CI pipeline that runs your tests on every push and on a nightly schedule.
- An API test suite with schema validation and mocking.
- AI-augmented workflow with prompt logs you can talk through in interviews.
- A 3-minute demo video and LinkedIn post about your journey.
- Confidence to apply for SDET / Automation Engineer roles.

Print this calendar. Tape it to your wall. Cross off each day. The compound effect of 90 hours of deliberate practice is bigger than you think.